

WebBuilder: So bauen wir den Builder

Tutorial für Jan · Stand: Oktober 2026

Hier steht Schritt für Schritt, wie wir den Builder in `builder/` fertig bauen. Am Ende kann man Text, Überschriften, Bilder und Buttons auf das Blatt setzen, anklicken, rechts bearbeiten, verschieben, löschen und alles als HTML-Datei herunterladen.

Wir brauchen dafür kein Framework, kein npm und keinen Build-Schritt. Es bleibt bei den Dateien, die es schon gibt: `index.html`, `style.css`, `colors.css` und `index.js`. Jeder Schritt läuft für sich, du kannst also nach jedem testen.

- | | |
|---|---|
| 1. Vorbereitung | 7. Schritt 4: Überschrift, Bild, Button |
| 2. Drei Regeln aus der Datenschutzerklärung | 8. Schritt 5: Verschieben und Löschen |
| 3. Die Grundidee | 9. Schritt 6: Export als HTML |
| 4. Schritt 1: HTML und CSS | 10. Wenn was nicht klappt |
| 5. Schritt 2: Text auf das Blatt | 11. Git |
| 6. Schritt 3: Bearbeiten im rechten Panel | 12. Ideen für danach |

Ganz hinten steht die fertige `index.js` komplett, zum Vergleichen.

1. Vorbereitung

- Erst `git pull`, damit du den aktuellen Stand hast. Das Design ist inzwischen schlicht und dunkel, `colors.css` und `style.css` haben sich dabei geändert.
- Zum Testen reicht es, `builder/index.html` direkt im Browser zu öffnen. Oder du startest im Projektordner einen kleinen Server und gehst auf `http://localhost:8000/builder/`:

```
python3 -m http.server 8000
```

- **F12** öffnet die Entwicklertools. Im Tab „Konsole“ stehen alle Fehler. Da zuerst schauen, wenn etwas nicht geht.
- Nach einer Änderung mit **F5** neu laden. Wenn sich nichts tut, **Strg+F5** (lädt ohne Cache).

2. Drei Regeln aus der Datenschutzerklärung

In der Datenschutzerklärung (Abschnitt 11 und 14) versprechen wir Folgendes, und der Code muss dazu passen:

1. **Alles bleibt im Arbeitsspeicher.** Kein `localStorage`, kein `sessionStorage`, keine IndexedDB, keine Cookies. Neu laden heißt: Entwurf weg. Das ist so gewollt.
2. **Nichts geht an den Server.** Kein `fetch()` oder `XMLHttpRequest` mit Inhalten aus dem Builder.
3. **Der Export entsteht im Browser** und wird direkt heruntergeladen.

Wenn wir davon abweichen wollen, müssen wir vorher die Datenschutzerklärung ändern.

Dazu kommt eine Sicherheitsregel: Text, den jemand eintippt, setzen wir immer mit `textContent` ein, nie mit `innerHTML`. Sonst wird ein eingetipptes `<b>` oder `<script>` als echtes HTML ausgeführt.

3. Die Grundidee

In den Kommentaren steht „json file“. Eine echte Datei brauchen wir nicht: Die ganze Seite steckt in einem JavaScript-Array namens `blocks`. Jeder Block ist ein Objekt mit einer `id`, einem `type` und den Feldern, die zu diesem Typ gehören:

```
blocks = [
  { id: 1, type: "heading", text: "Willkommen" },
  { id: 2, type: "text", text: "Hallo, ich bin Jan." },
  { id: 3, type: "image", src: "https://example.com/bild.jpg", alt: "Ein Bild" },
  { id: 4, type: "button", text: "Mehr", href: "https://example.com" },
];
```

Dazu kommen zwei Variablen: `selectedId` merkt sich, welcher Block gerade angeklickt ist. `nextId` zählt hoch, damit jeder Block eine eigene Nummer bekommt.

Jede Aktion im Builder läuft gleich ab:

Klick auf einen Button → `blocks` ändern → `updateCanvas()` → Blatt neu gezeichnet

Die wichtigste Regel im ganzen Builder: Wir ändern nie direkt das Blatt. Wir ändern nur `blocks` und lassen `updateCanvas()` das Blatt danach komplett neu zeichnen. So können Daten und Anzeige nie auseinanderlaufen, und der Export kann einfach `blocks` lesen.

4. Schritt 1: HTML und CSS

Ersetze `builder/index.html` komplett durch diese Version:

```
<!doctype html>
<html lang="de">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>WebBuilder F&J</title>

    <!-- Link für die Css files-->
    <link rel="stylesheet" href="colors.css" />
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <div class="builder-app">
      <!-- linkes Panel-->
      <aside class="sidebar sidebar-left">
        <button class="btn-add" onclick="AddText()">Text Hinzufügen</button>
        <button class="btn-add" onclick="addHeading()">Überschrift Hinzufügen</button>
        <button class="btn-add" onclick="addImage()">Bild Hinzufügen</button>
        <button class="btn-add" onclick="addButton()">Button Hinzufügen</button>
        <button class="btn-add" onclick="wipeCanvas()">Canvas Wipen</button>
        <button class="btn-add" onclick="exportHtml()">Als HTML exportieren</button>
      </aside>

      <!-- canvas (center)-->
      <main class="canvas-viewport">
        <div class="canvas-page" id="canvasPage"></div>
      </main>
    </div>
  </body>
</html>
```

```

        <!-- rechtes Panel-->
        <aside class="sidebar sidebar-right">
            <h2>Block</h2>
            <div id="props"></div>
            <button class="btn-add" onclick="moveSelected(-1)">Nach oben</button>
            <button class="btn-add" onclick="moveSelected(1)">Nach unten</button>
            <button class="btn-add" onclick="deleteSelected()">Löschen</button>
        </aside>
    </div>
    <script src="index.js"></script>
</body>
</html>

```

Neu sind die Buttons links, das leere Blatt (der Testtext fliegt raus, weil der Inhalt jetzt aus `blocks` kommt) und das rechte Panel. Das Grid in `style.css` hat die rechte Spalte schon, sie war bisher nur leer.

Buttons, deren Funktion erst in einem späteren Schritt kommt, werfen bis dahin beim Klicken einen Fehler in der Konsole. Das ist normal.

Häng das hier unten an `builder/style.css` an:

```

/* Bilder und Buttons im Canvas */
.canvas-page {
    color-scheme: light; /* Links im weißen Blatt normal blau */
}

.canvas-page img {
    max-width: 100%;
    align-self: flex-start;
}

.canvas-page a {
    align-self: flex-start;
}

textarea.form-control {
    min-height: 80px;
    resize: vertical;
}

```

- `color-scheme: light` sorgt dafür, dass Links auf dem weißen Blatt normal blau sind, obwohl der Rest dunkel ist.
- `align-self: flex-start` verhindert, dass Bilder und Links im Blatt auf die volle Breite gezogen werden.

5. Schritt 2: Text auf das Blatt

Ersetze den ganzen Inhalt von `builder/index.js` durch das hier:

```

// Der ganze Inhalt der Seite. Jeder Block ist ein Objekt.
let blocks = [];
let selectedId = null;
let nextId = 1;

// Zeichnet den Canvas komplett neu aus "blocks"
function updateCanvas() {

```

```

const page = document.getElementById("canvasPage");
page.replaceChildren();

for (const block of blocks) {
  const el = renderBlock(block);
  el.classList.add("canvas-block");
  if (block.id === selectedId) {
    el.classList.add("selected");
  }
  el.onclick = (event) => {
    event.preventDefault(); // Links im Canvas nicht öffnen
    selectBlock(block.id);
  };
  page.append(el);
}

// Baut das HTML-Element für einen Block
function renderBlock(block) {
  const p = document.createElement("p");
  p.textContent = block.text;
  return p;
}

function getSelected() {
  return blocks.find((block) => block.id === selectedId);
}

function selectBlock(id) {
  selectedId = id;
  updateCanvas();
  updateProps();
}

// Kommt in Schritt 3
function updateProps() {}

function addBlock(block) {
  block.id = nextId++;
  blocks.push(block);
  selectBlock(block.id);
}

function AddText() {
  addBlock({ type: "text", text: "Neuer Text" });
}

function wipeCanvas() {
  if (!confirm("Wirklich alles löschen?")) return;
  blocks = [];
  selectBlock(null);
}

// Beim Laden einmal alles zeichnen
updateCanvas();
updateProps();

```

- `updateCanvas()` leert das Blatt mit `replaceChildren()` und baut für jeden Block ein neues Element.

- `renderBlock()` macht aus einem Block ein HTML-Element. Erstmal kann es nur Text, in Schritt 4 kommen die anderen Typen dazu.
- Beim Klick auf einen Block merkt sich `selectBlock()` die id und zeichnet neu. Der angeklickte Block bekommt die Klasse `selected`, für die es in `style.css` schon den blauen Rahmen gibt.
- `event.preventDefault()` brauchen wir ab Schritt 4: Es verhindert, dass ein Klick auf einen Link im Blatt die Seite verlässt.
- `addBlock()` ist die einzige Stelle, an der neue Blöcke entstehen: id vergeben, ans Ende von `blocks` hängen, auswählen.
- Die zwei Zeilen ganz unten zeichnen beim Laden einmal alles. **Die bleiben immer ganz unten in der Datei**, neuer Code kommt darüber.

Testen: Ein paar Mal auf „Text Hinzufügen“ klicken. Es erscheinen Absätze „Neuer Text“, der neueste hat einen blauen Rahmen. Ein Klick auf einen anderen Absatz verschiebt den Rahmen. „Canvas Wipen“ fragt nach und leert dann das Blatt.

6. Schritt 3: Bearbeiten im rechten Panel

Füg diese Konstante oben direkt unter die drei Variablen ein:

```
// Beschriftungen für die Eingabefelder rechts
const LABELS = {
  text: "Text",
  src: "Bild-URL",
  alt: "Beschreibung (alt)",
  href: "Link-Ziel",
};
```

Ersetze die leere `updateProps()` durch diese zwei Funktionen:

```
// Zeichnet die Eingabefelder rechts für den ausgewählten Block
function updateProps() {
  const props = document.getElementById("props");
  props.replaceChildren();

  const block = getSelected();
  if (!block) {
    props.textContent = "Kein Block ausgewählt.";
    return;
  }

  for (const key of Object.keys(block)) {
    if (key === "id" || key === "type") continue;
    props.append(makeField(block, key));
  }
}

function makeField(block, key) {
  const group = document.createElement("div");
  group.className = "form-group";

  const label = document.createElement("label");
  label.textContent = LABELS[key];

  const input = document.createElement(key === "text" ? "textarea" : "input");
```

```

input.className = "form-control";
input.value = block[key];
input.oninput = () => {
    block[key] = input.value;
    updateCanvas(); // nur den Canvas, sonst verliert das Feld den Fokus
};

group.append(label, input);
return group;
}

```

- `updateProps()` baut für jedes Feld des ausgewählten Blocks ein Eingabefeld. `id` und `type` überspringen wir, die soll man nicht ändern können.
- Für Text gibt es ein großes Feld (`textarea`), für alles andere ein normales `input`.
- Bei jedem Tastendruck (`oninput`) schreiben wir den neuen Wert in den Block und zeichnen das Blatt neu.
- **Wichtig:** Dort nur `updateCanvas()` aufrufen, nicht `updateProps()`. Sonst wird das Eingabefeld bei jedem Buchstaben neu gebaut und du kannst nicht weitertippen.

Testen: Einen Block anklicken und rechts im Feld tippen. Der Text im Blatt ändert sich sofort mit. Tipp auch mal `<b>hallo</b>` ein: Das muss genau so als Text dastehen und darf nicht fett werden.

7. Schritt 4: Überschrift, Bild, Button

Ersetze `renderBlock()` durch diese Version:

```

// Baut das HTML-Element für einen Block
function renderBlock(block) {
    if (block.type === "heading") {
        const h = document.createElement("h2");
        h.textContent = block.text;
        return h;
    }
    if (block.type === "image") {
        if (!block.src) {
            const p = document.createElement("p");
            p.textContent = "[Bild: rechts eine URL eintragen]";
            return p;
        }
        const img = document.createElement("img");
        img.src = block.src;
        img.alt = block.alt;
        return img;
    }
    if (block.type === "button") {
        const a = document.createElement("a");
        a.textContent = block.text;
        a.href = block.href;
        return a;
    }
    // type "text"
    const p = document.createElement("p");
    p.textContent = block.text;
    return p;
}

```

Und füg diese drei Funktionen direkt unter `AddText()` ein:

```
function addHeading() {
  addBlock({ type: "heading", text: "Neue Überschrift" });
}

function addImage() {
  addBlock({ type: "image", src: "", alt: "" });
}

function addButton() {
  addBlock({ type: "button", text: "Klick mich", href: "https://" });
}
```

- Jeder Typ hat seine eigenen Felder: Text hat `text`, Bild hat `src` und `alt`, Button hat `text` und `href`. Das rechte Panel passt sich von selbst an, weil `updateProps()` einfach alle Felder des Blocks durchgeht.
- Ein Bild ohne URL wäre unsichtbar, und man könnte es nicht anklicken. Deshalb zeigen wir dann einen Platzhaltertext.
- Der „Button“ ist erstmal ein normaler Link. Wie er wie ein Button aussieht, steht bei den Ideen am Ende.

Testen: „Bild Hinzufügen“, dann rechts eine Bild-URL einfügen (z. B. auf Wikipedia Rechtsklick auf ein Bild → „Bildadresse kopieren“). „Button Hinzufügen“ und den Button im Blatt anklicken: Er wird ausgewählt, und die Seite bleibt offen.

8. Schritt 5: Verschieben und Löschen

Füg diese zwei Funktionen über den zwei Zeilen ganz unten ein:

```
function moveSelected(direction) {
  const i = blocks.findIndex((block) => block.id === selectedId);
  const j = i + direction;
  if (i === -1 || j < 0 || j >= blocks.length) return;

  [blocks[i], blocks[j]] = [blocks[j], blocks[i]];
  updateCanvas();
}

function deleteSelected() {
  blocks = blocks.filter((block) => block.id !== selectedId);
  selectBlock(null);
}
```

- `findIndex()` sucht die Position des ausgewählten Blocks. `direction` ist `-1` (nach oben) oder `1` (nach unten).
- Die Zeile mit `[a, b] = [b, a]` tauscht zwei Einträge im Array.
- Ist nichts ausgewählt oder der Block schon ganz oben oder unten, macht die Funktion einfach nichts.
- `deleteSelected()` behält alle Blöcke außer dem ausgewählten und hebt die Auswahl auf.

Testen: Ein paar verschiedene Blöcke anlegen, mit „Nach oben“ und „Nach unten“ sortieren, einen löschen.

9. Schritt 6: Export als HTML

Füg diese drei Funktionen über den zwei Zeilen ganz unten ein:

```
// Macht Text sicher für HTML: aus < wird &lt; usw.
function escapeHtml(text) {
  return text
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll("'", "&quot;");
}

function blockToHtml(block) {
  if (block.type === "heading") {
    return `

## ${escapeHtml(block.text)}</h2>`; } if (block.type === "image") { return `${escapeHtml(block.alt)}${escapeHtml(block.text)}</p>`; } function exportHtml() { const body = blocks.map(blockToHtml).join("\n"); const html = `<!DOCTYPE html> <html lang="de"> <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>Meine Seite</title> </head> <body> ${body} </body> </html> `; // Datei im Browser bauen und herunterladen, ohne Server const file = new Blob([html], { type: "text/html" }); const link = document.createElement("a"); link.href = URL.createObjectURL(file); link.download = "seite.html"; link.click(); }


```

- `blockToHtml()` macht aus jedem Block eine Zeile HTML. Hier bauen wir HTML als Text zusammen, deshalb muss alles, was jemand eingetippt hat, durch `escapeHtml()`. Sonst macht ein `<` oder `"` im Text die exportierte Seite kaputt.
- In `escapeHtml()` muss `&` als Erstes ersetzt werden. Sonst würde aus einem schon ersetzten `<` danach `&lt;`.

- `exportHtml()` setzt alles in ein vollständiges HTML-Gerüst.
- `new Blob(...)` macht daraus eine Datei im Speicher, `URL.createObjectURL()` gibt ihr eine Adresse, und ein unsichtbarer Link mit `download` lädt sie herunter. Der Server bekommt davon nichts mit.

Testen: Ein paar Blöcke anlegen, auf „Als HTML exportieren“ klicken und die heruntergeladene `seite.html` öffnen. Alles sollte da sein, in der richtigen Reihenfolge.

10. Wenn was nicht klappt

Problem	Ursache und Lösung
Klick auf einen Button tut nichts	F12 → Konsole. „... is not defined“ heißt meistens Tippfehler oder falsche Groß- und Kleinschreibung (<code>AddText</code> und <code>addText</code> sind zwei verschiedene Namen).
Änderungen tauchen nicht auf	Datei gespeichert? Dann <code>Strg+F5</code> .
Feld rechts verliert beim Tippen den Fokus	In <code>oninput</code> wird <code>updateProps()</code> aufgerufen. Dort nur <code>updateCanvas()</code> .
Eingetippter Text wird fett oder als HTML angezeigt	Irgendwo steht <code>innerHTML</code> statt <code>textContent</code> .
Klick auf einen Button im Blatt öffnet eine andere Seite	<code>event.preventDefault()</code> in <code>updateCanvas()</code> fehlt.
Bild ist riesig	Das CSS aus Schritt 1 fehlt (<code>max-width: 100%</code>).
„Cannot access 'LABELS' before initialization“	<code>LABELS</code> steht unter den zwei Zeilen ganz unten. Es muss oben bei den Variablen stehen.

11. Git

Am besten nach jedem Schritt einen Commit machen. Dann kommt man leicht zurück, wenn etwas kaputtgeht:

```
git pull
git add builder/
git commit -m "Builder: Schritt 2, Text auf das Blatt"
git push
```

Vor dem Arbeiten immer erst `git pull`, weil wir beide auf `main` pushen.

12. Ideen für danach

- **Buttons im Export wie Buttons aussehen lassen:** In `exportHtml()` einen `<style>`-Block in den `<head>` schreiben, z. B. für `a` mit Rahmen und Innenabstand.
- **Mehr Felder**, z. B. Bildbreite oder Textausrichtung: im Objekt in `addImage()` usw. ergänzen, in `renderBlock()` und `blockToHtml()` benutzen und in `LABELS` eine Beschriftung eintragen. Das rechte Panel zeigt das neue Feld dann von selbst an.

- **Entwurf speichern und laden:** `JSON.stringify(blocks)` genauso wie beim Export als Datei herunterladen und mit `<input type="file">` wieder einlesen. Das passt zur Datenschutzerklärung, weil die Datei nur auf dem Gerät landet. `localStorage` wäre bequemer, verstößt aber gegen unsere Regeln.
- **Zeilenumbrüche im Text:** Im Moment werden sie zu Leerzeichen. Mit `white-space: pre-wrap` im CSS bleiben sie erhalten.
- **Drag & Drop** zum Sortieren statt der zwei Buttons.

Anhang: Die fertige index.js

So sieht `builder/index.js` nach Schritt 6 aus, wenn du alles wie beschrieben eingefügt hast.

```
// Der ganze Inhalt der Seite. Jeder Block ist ein Objekt.
let blocks = [];
let selectedId = null;
let nextId = 1;

// Beschriftungen für die Eingabefelder rechts
const LABELS = {
  text: "Text",
  src: "Bild-URL",
  alt: "Beschreibung (alt)",
  href: "Link-Ziel",
};

// Zeichnet den Canvas komplett neu aus "blocks"
function updateCanvas() {
  const page = document.getElementById("canvasPage");
  page.replaceChildren();

  for (const block of blocks) {
    const el = renderBlock(block);
    el.classList.add("canvas-block");
    if (block.id === selectedId) {
      el.classList.add("selected");
    }
    el.onclick = (event) => {
      event.preventDefault(); // Links im Canvas nicht öffnen
      selectBlock(block.id);
    };
    page.append(el);
  }
}

// Baut das HTML-Element für einen Block
function renderBlock(block) {
  if (block.type === "heading") {
    const h = document.createElement("h2");
    h.textContent = block.text;
    return h;
  }
  if (block.type === "image") {
    if (!block.src) {
      const p = document.createElement("p");
      p.textContent = "[Bild: rechts eine URL eintragen]";
      return p;
    }
    const img = document.createElement("img");
    img.src = block.src;
    img.alt = block.alt;
    return img;
  }
  if (block.type === "button") {
    const a = document.createElement("a");
    a.textContent = block.text;
    a.href = block.href;
    return a;
  }
  // type "text"
}
```

```

    const p = document.createElement("p");
    p.textContent = block.text;
    return p;
}

function getSelected() {
    return blocks.find((block) => block.id === selectedId);
}

function selectBlock(id) {
    selectedId = id;
    updateCanvas();
    updateProps();
}

// Zeichnet die Eingabefelder rechts für den ausgewählten Block
function updateProps() {
    const props = document.getElementById("props");
    props.replaceChildren();

    const block = getSelected();
    if (!block) {
        props.textContent = "Kein Block ausgewählt.";
        return;
    }

    for (const key of Object.keys(block)) {
        if (key === "id" || key === "type") continue;
        props.append(makeField(block, key));
    }
}

function makeField(block, key) {
    const group = document.createElement("div");
    group.className = "form-group";

    const label = document.createElement("label");
    label.textContent = LABELS[key];

    const input = document.createElement(key === "text" ? "textarea" : "input");
    input.className = "form-control";
    input.value = block[key];
    input.oninput = () => {
        block[key] = input.value;
        updateCanvas(); // nur den Canvas, sonst verliert das Feld den Fokus
    };

    group.append(label, input);
    return group;
}

function addBlock(block) {
    block.id = nextId++;
    blocks.push(block);
    selectBlock(block.id);
}

function AddText() {
    addBlock({ type: "text", text: "Neuer Text" });
}

function addHeading() {

```

```

    addBlock({ type: "heading", text: "Neue Überschrift" });
  }

function addImage() {
  addBlock({ type: "image", src: "", alt: "" });
}

function addButton() {
  addBlock({ type: "button", text: "Klick mich", href: "https://" });
}

function wipeCanvas() {
  if (!confirm("Wirklich alles löschen?")) return;
  blocks = [];
  selectBlock(null);
}

function moveSelected(direction) {
  const i = blocks.findIndex((block) => block.id === selectedId);
  const j = i + direction;
  if (i === -1 || j < 0 || j >= blocks.length) return;

  [blocks[i], blocks[j]] = [blocks[j], blocks[i]];
  updateCanvas();
}

function deleteSelected() {
  blocks = blocks.filter((block) => block.id !== selectedId);
  selectBlock(null);
}

// Macht Text sicher für HTML: aus < wird &lt; usw.
function escapeHtml(text) {
  return text
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll("'", "&quot;");
}

function blockToHtml(block) {
  if (block.type === "heading") {
    return `

## ${escapeHtml(block.text)}</h2>`; } if (block.type === "image") { return `${escapeHtml(block.alt)}${escapeHtml(block.text)}</p>`; } function exportHtml() { const body = blocks.map(blockToHtml).join("\n"); const html = `<!DOCTYPE html> <html lang="de"> <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>Meine Seite</title> </head>


```

```
<body>
${body}
</body>
</html>
`;

// Datei im Browser bauen und herunterladen, ohne Server
const file = new Blob([html], { type: "text/html" });
const link = document.createElement("a");
link.href = URL.createObjectURL(file);
link.download = "seite.html";
link.click();
}

// Beim Laden einmal alles zeichnen
updateCanvas();
updateProps();
```